SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

SSAFY

SW문제해결 기본

Tree

Python

학습시작

자료구조를 이용해 가계도를 표현해 봅시다!

1. 부모와 자식 관계를 표현할 수 있는 [ ]자료구조가 필요합니다.
2. [ ] 자료구조인 [ ]를 이용해 표현할 수 있습니다.

- 현실 세계를 표현할 수 있는 자료구조를 배울 수 있습니다.
- 1 : N 관계를 자료구조로 표현할 수 있게 됩니다.
- 파일시스템, CPU 스케줄링, 관계형 데이터베이스의 인덱스에 대해 이해하는 데 도움이 됩니다.

Objectives

학습목표

- 트리 자료구조의 개념과 기본 연산을 설명할 수 있다.
- 이진 트리에 대해 설명하고 이를 구현할 수 있다.
- 이진 트리의 순회 방법을 설명할 수 있다.
- 힙에 대해 이해하고 기본 연산을 설명할 수 있다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

목 차

Tree

- 트리
- 이진 트리
- 이진 트리의 표현1
- 이진 트리의 표현2
- 순회
- 연습 문제
- 이진 탐색 트리
- 힙

Tree

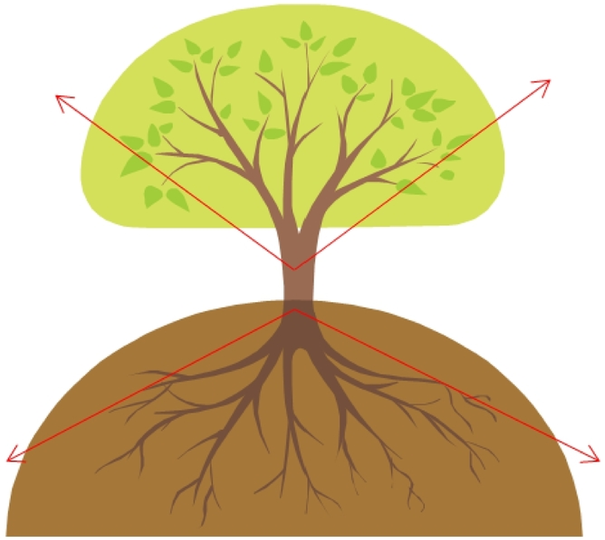
SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

트리

트리의 개념

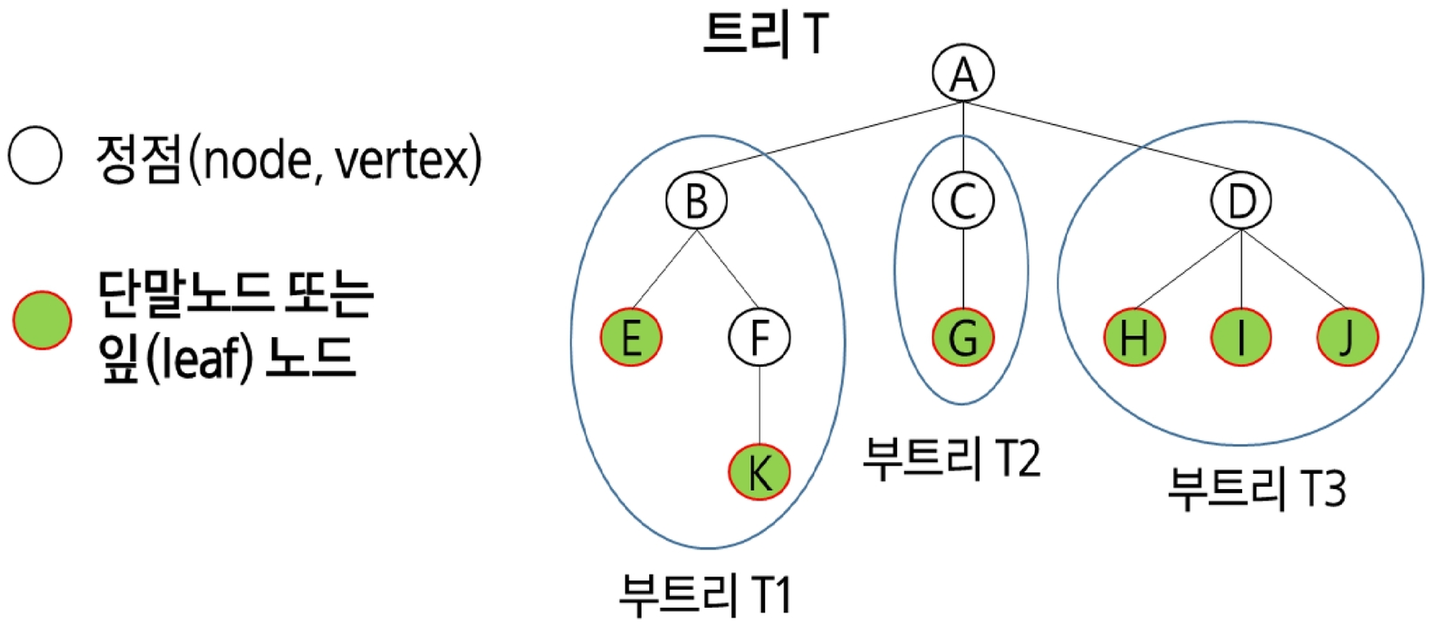
- 비선형 자료구조
- 원소들 간에 1:N 관계를 가지는 자료구조
- 원소들 간에 계층관계를 가지는 계층형 자료구조
- 상위 원소에서 하위 원소로 내려가면서 확장되는 트리(나무) 모양의 구조



<그림1_SW문제해결 기본_Tree_트리>

트리의 정의

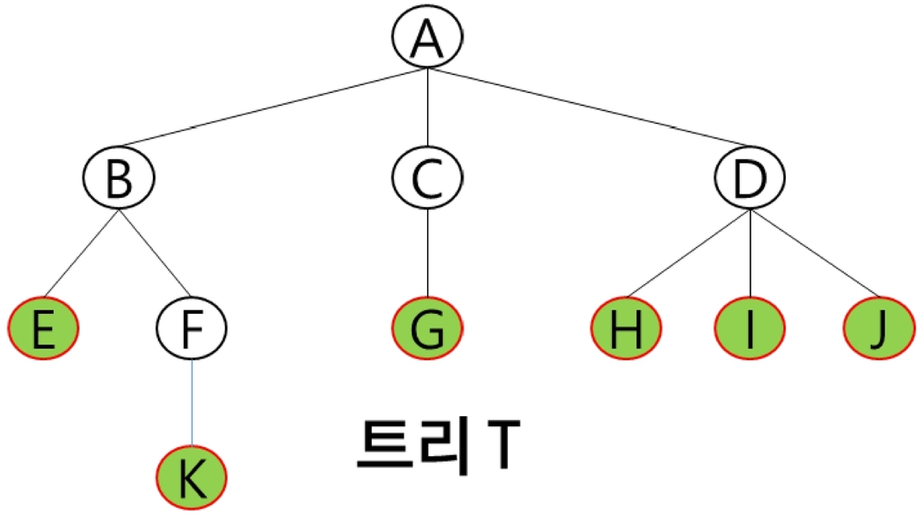
- 한 개 이상의 노드로 이루어진 유한 집합이며 다음 조건을 만족한다.
  - 노드 중 최상위 노드를 루트(root)라 한다.
  - 나머지 노드들은 n(>=0)개의 부분 집합 T1, …, TN으로 분리될 수 있다.
- 이들 T1, …, TN은 각각 하나의 트리가 되며(재귀적 정의) 부트리(subtree)라 한다.



<그림2_SW문제해결 기본_Tree_트리의 정의>

용어정리

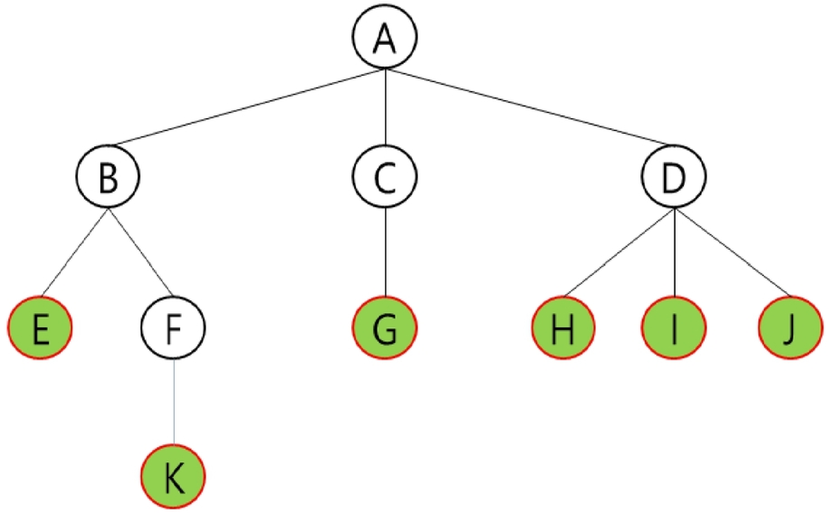
- 노드(node) - 트리의 원소
  - 트리 T의 노드 - A, B, C, D, E, F, G, H, I, J, K
- 간선(edge) - 노드를 연결하는 선. 부모 노드와 자식 노드를 연결
- 루트 노드(root node) - 트리의 시작 노드
  - 트리 T의 루트 노드 - A



<그림3_SW문제해결 기본_Tree_용어정리 1>

용어정리

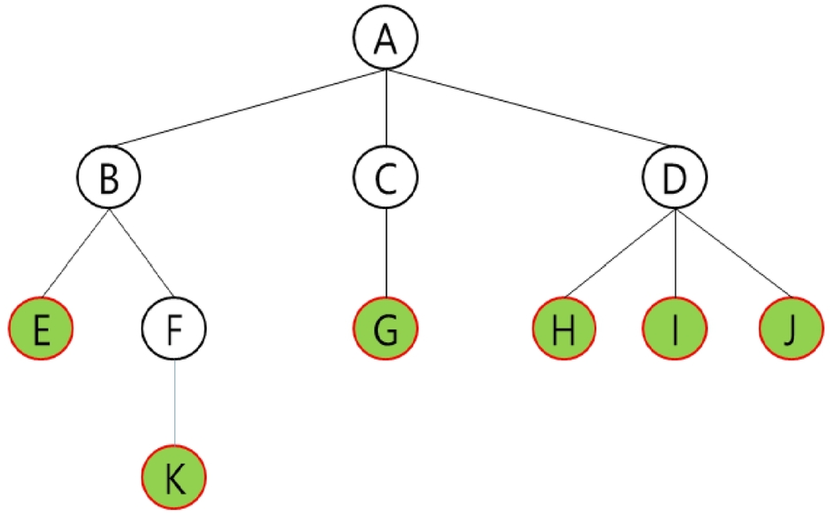
- 형제 노드(sibling node)
  - 같은 부모 노드의 자식 노드들
  - B, C, D 는 형제 노드
- 조상 노드
  - 간선을 따라 루트 노드까지 이르는 경로에 있는 모든 노드들
  - K의 조상 노드 : F, B, A
- 서브 트리(subtree)
  - 부모 노드와 연결된 간선을 끊었을 때 생성되는 트리
- 자손 노드
  - 서브 트리에 있는 하위 레벨의 노드들
  - B의 자손 노드 - E, F, K



<그림4_SW문제해결 기본_Tree_용어정리 2>

용어정리

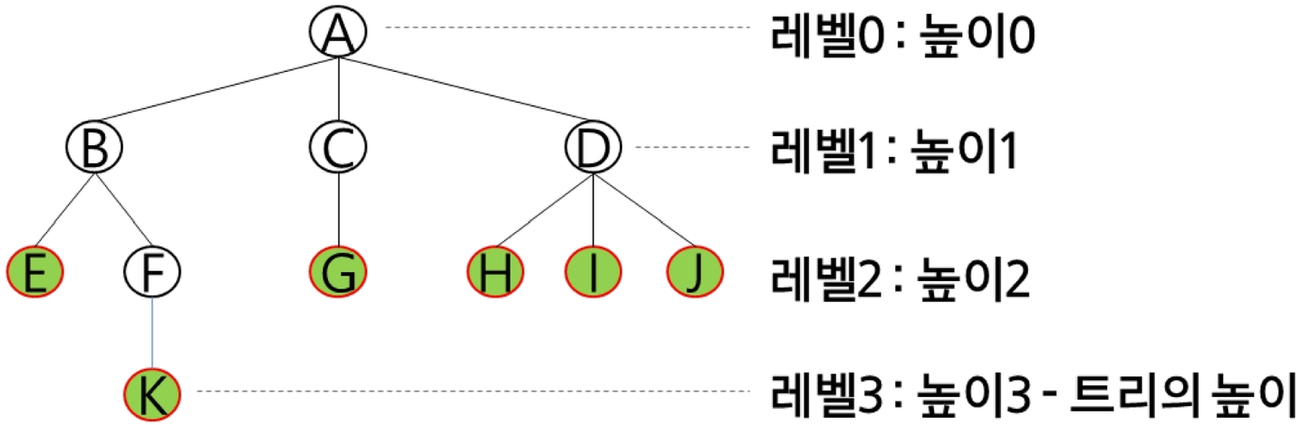
- 노드의 차수
  - 노드에 연결된 자식 노드의 수
  - B의 차수 = 2, C의 차수 = 1
- 트리의 차수
  - 트리에 있는 노드의 차수 중에서 가장 큰 값
  - 트리 T의 차수 = 3
- 단말 노드 (리프 노드)
  - 차수가 0인 노드. 자식 노드가 없는 노드



<그림5_SW문제해결 기본_Tree_용어정리 3>

용어정리

- 노드의 높이
  - 루트에서 노드에 이르는 간선의 수. 노드의 레벨
  - B의 높이 = 1, F의 높이 = 2
- 트리의 높이
  - 트리에 있는 노드의 높이 중에서 가장 큰 값. 최대 레벨
  - 트리 T의 높이 = 3



<그림6_SW문제해결 기본_Tree_용어정리 4>

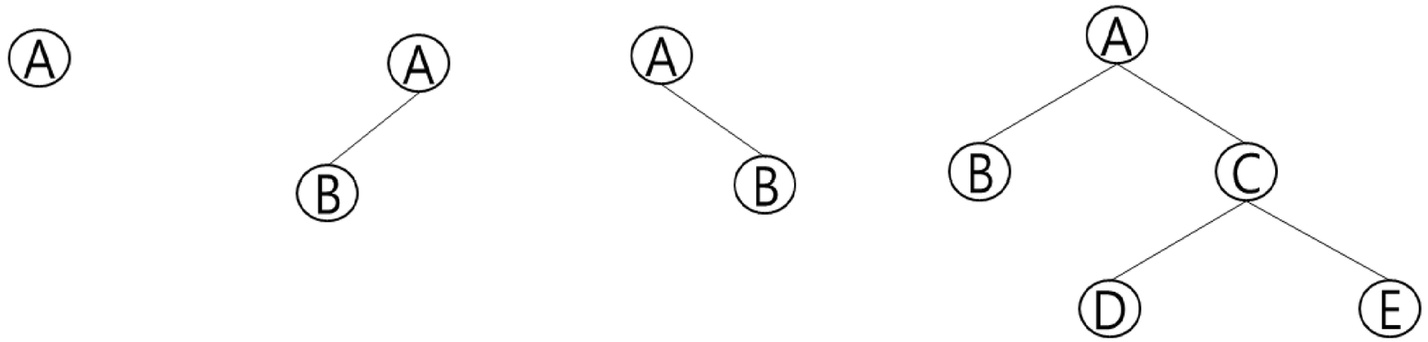
SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

이진 트리

이진 트리

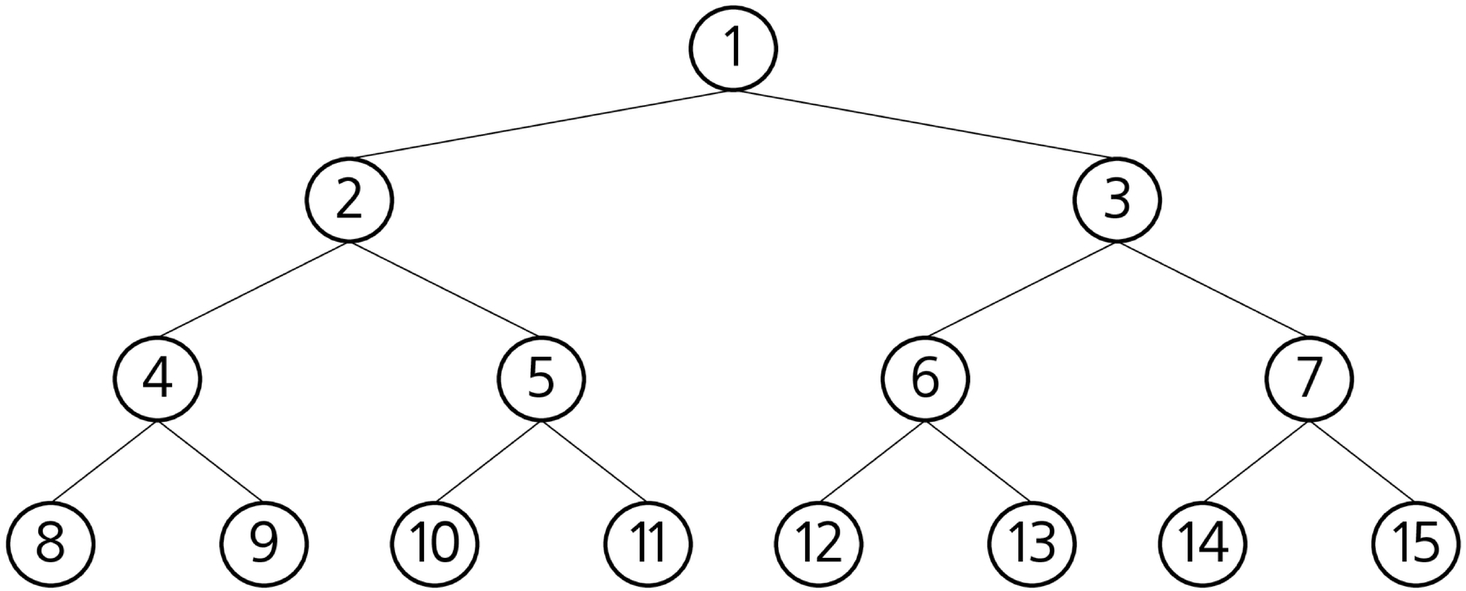
- 모든 노드들이 2개 이내의 서브 트리를 갖는 특별한 형태의 트리
- 각 노드가 자식 노드를 최대한 2개 까지만 가질 수 있음
  - 왼쪽 자식 노드 (left child node)
  - 오른쪽 자식 노드 (right child node)
- 이진 트리의 예



<그림7_SW문제해결 기본_Tree_이진 트리의 예>

이진 트리의 특성

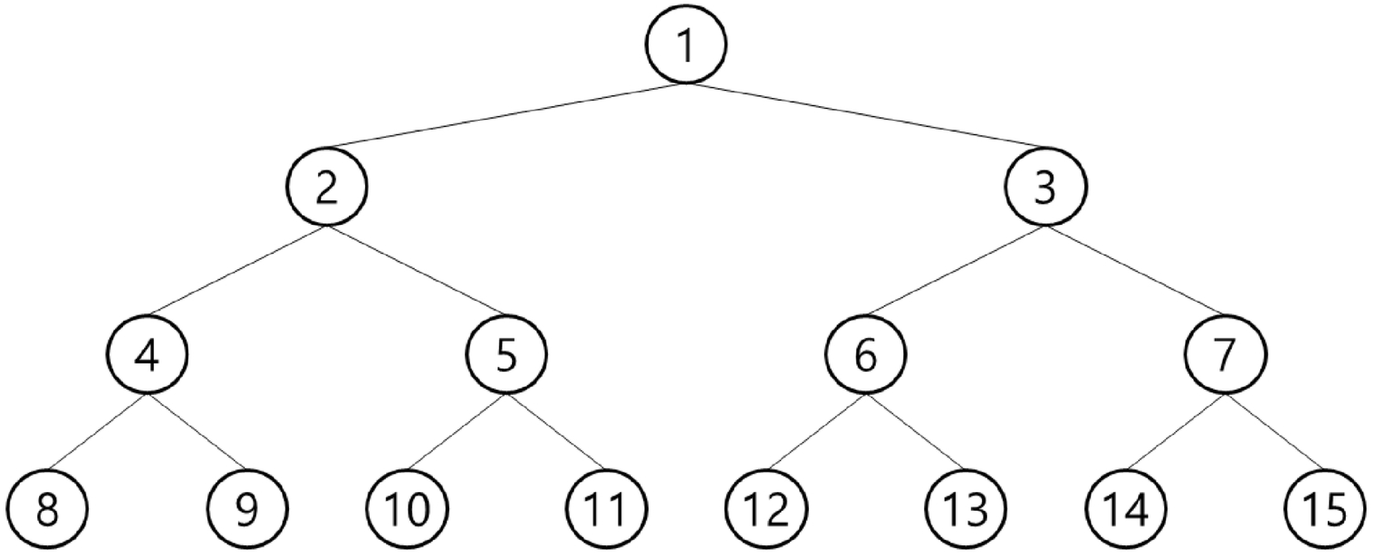
- 레벨 i에서의 노드의 최대 개수는 2^i 개
- 높이가 h인 이진 트리가 가질 수 있는 노드의 최소 개수는 (h+1)개가 되며, 최대 개수는 $(2^{h+1}-1)$개



<그림8_SW문제해결 기본_Tree_이진 트리>

포화 이진 트리 (Full Binary Tree)

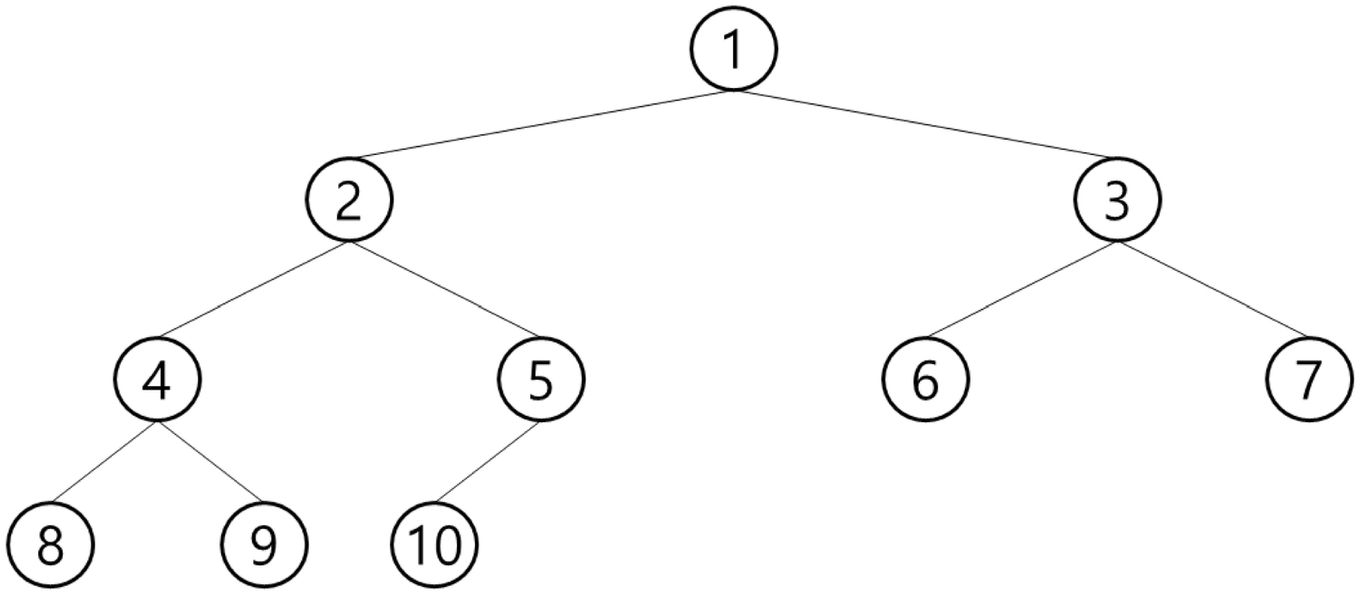
- 모든 레벨에 노드가 포화상태로 차 있는 이진 트리
- 높이가 h일 때, 최대의 노드의 개수인 $(2^{h+1}-1)$의 노드를 가진 이진 트리
  - 높이 3일 때 $2^{3+1}-1 = 15$개의 노드를 갖습니다.
- 루트를 1번으로 하여 $2^{h+1}-1$까지 정해진 위치에 대한 노드 번호를 가짐



<그림9_SW문제해결 기본_Tree_포화 이진 트리>

완전 이진 트리 (Complete Binary Tree)

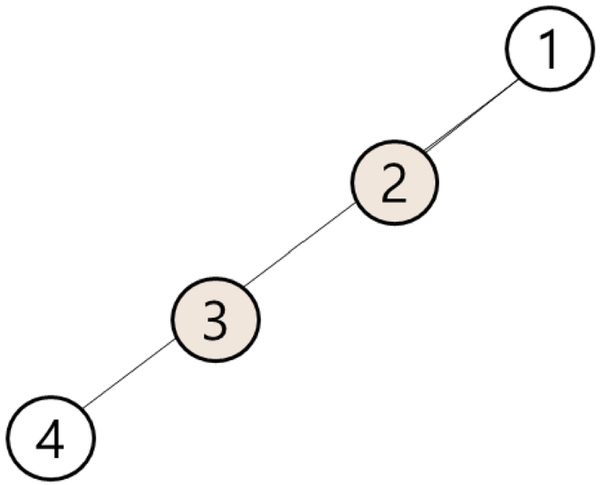
- 높이가 h이고 노드 수가 n개일 때 (단, $2^h \leq n \leq 2^{h+1}-1$),
- 포화 이진 트리의 노드 번호 1번부터 n번까지 빈 자리가 없는 이진 트리
  - 예) 노드가 10개인 완전 이진 트리



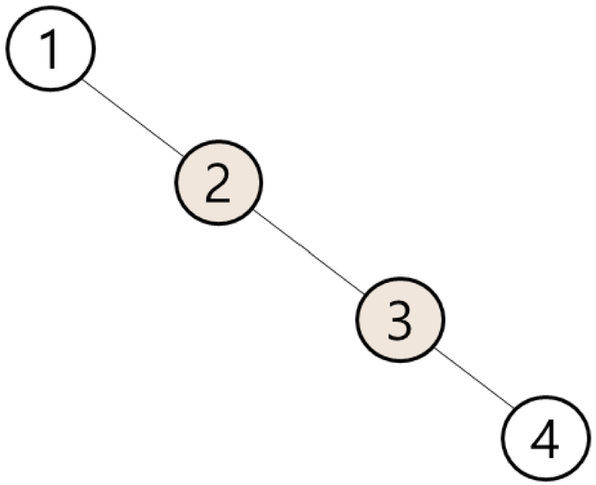
<그림10_SW문제해결 기본_Tree_노드가 10개인 완전 이진 트리>

편향 이진 트리 (Skewed Binary Tree)

- 높이가 h에 대한 최소 개수의 노드를 가지면서 한쪽 방향의 자식 노드만을 가진 이진 트리
  - 왼쪽 편향 이진 트리
  - 오른쪽 편향 이진 트리



<그림11_SW문제해결 기본_Tree_왼쪽 편향 이진 트리>



<그림12_SW문제해결 기본_Tree_오른쪽 편향 이진 트리>

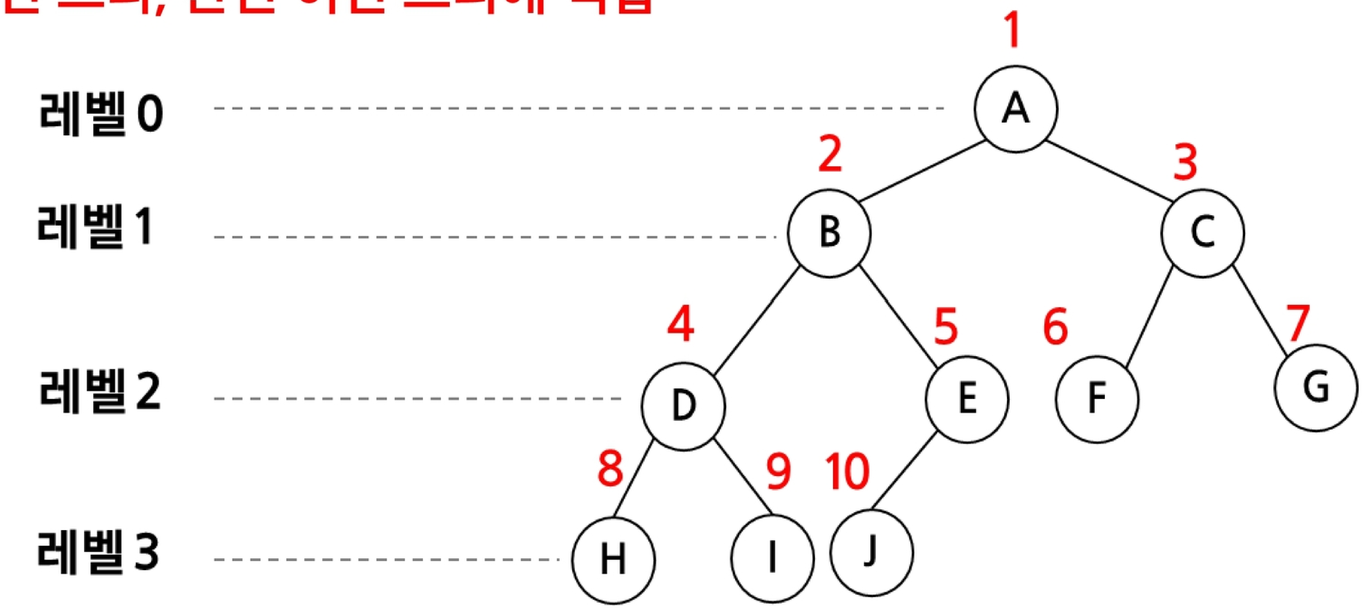
SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

이진 트리 표현 1

배열을 이용한 이진 트리의 표현

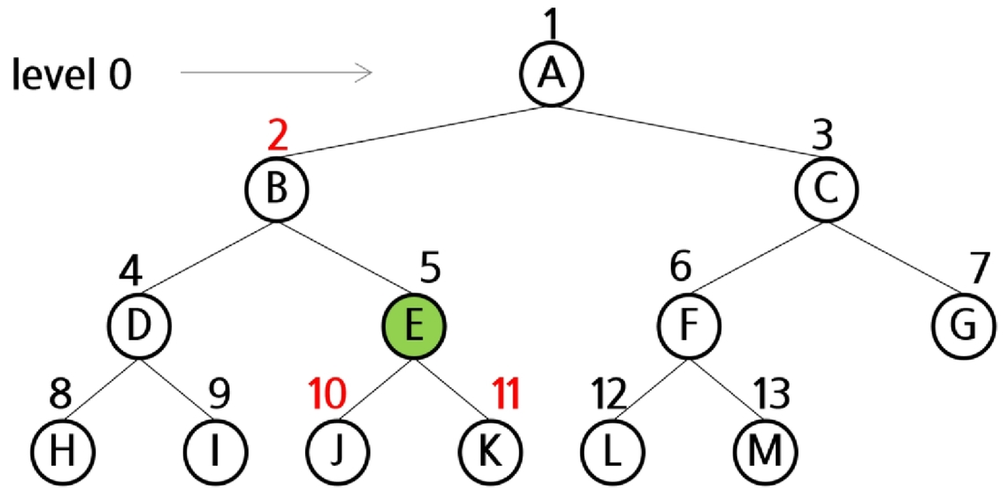
- 이진 트리에 각 노드 번호를 다음과 같이 부여
- 루트의 번호를 1번으로 부여
- 레벨 n에 있는 노드에 대하여 왼쪽부터 오른쪽으로 2^n 부터 $2^{n+1}-1$ 까지 번호를 차례로 부여
- 포화 이진 트리, 완전 이진 트리에 적합



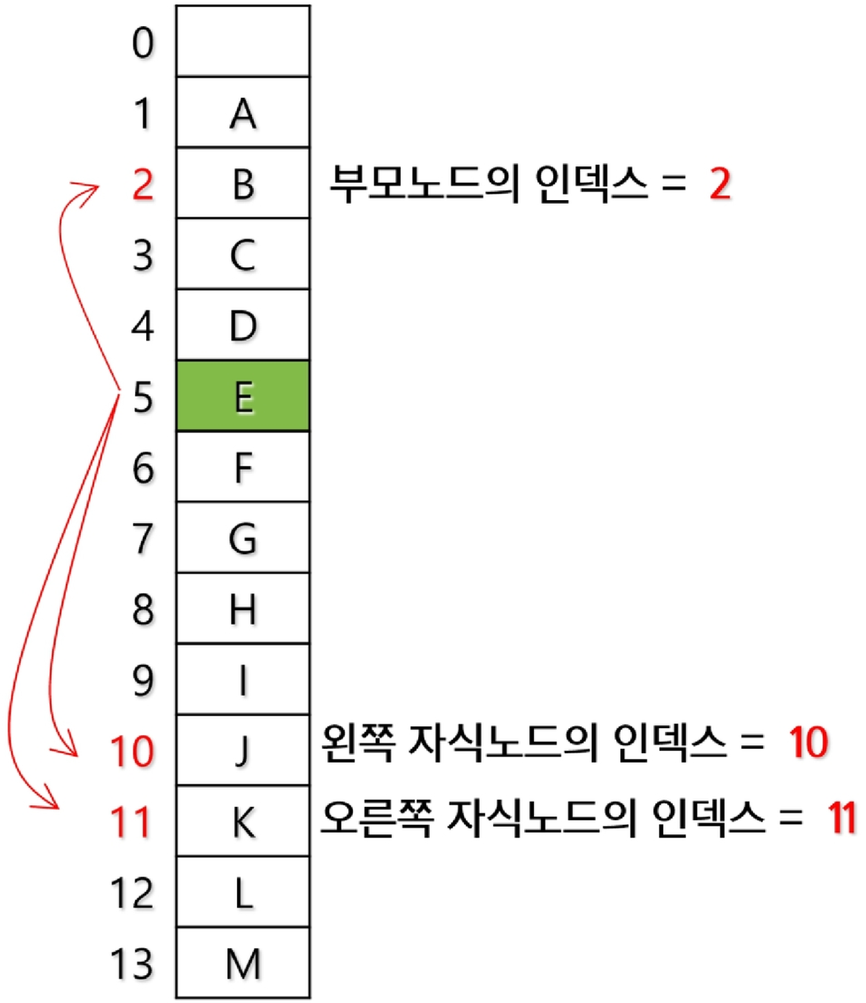
<그림18_SW문제해결 기본_Tree_배열을 이용한 이진 트리의 표현>

노드 번호의 성질

- 노드 번호가 i인 노드의 부모 노드 번호? i//2
- 노드 번호가 i인 노드의 왼쪽 자식 노드 번호? 2*i
- 노드 번호가 i인 노드의 오른쪽 자식 노드 번호? 2*i + 1
- 레벨 n의 노드 번호 시작 번호는? 2^n



<그림19_SW문제해결 기본_Tree_노드 번호의 성질 1>



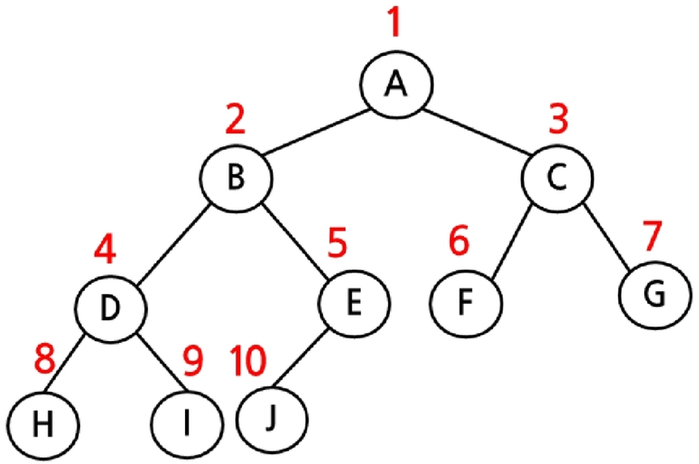
<그림20_SW문제해결 기본_Tree_노드 번호의 성질 2>

배열을 이용한 이진 트리의 표현

- 노드 번호를 배열의 인덱스로 사용
- 높이가 h인 이진 트리를 위한 배열의 크기는?
  - 레벨 i의 최대 노드 수는? 2^i
  - 따라서 $1 + 2 + 4 + 8 \cdots + 2^i = \sum 2^i = 2^{h+1}-1$

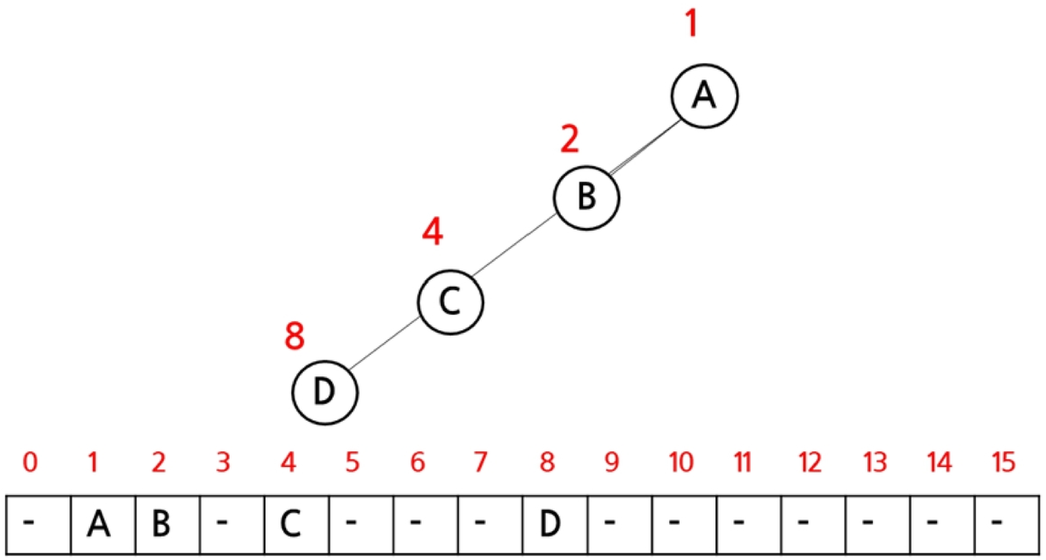
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
-	A	B	C	D	E	F	G	H	I	J	-	-	-	-	-

<그림21_SW문제해결 기본_Tree_배열을 이용한 이진 트리의 표현 1>

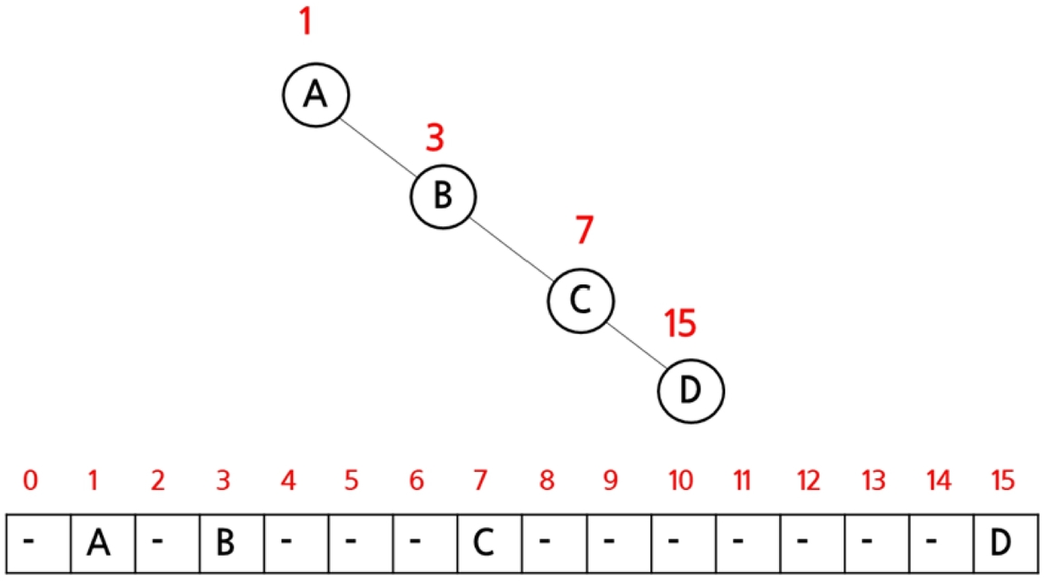


<그림22_SW문제해결 기본_Tree_배열을 이용한 이진 트리의 표현 2>

배열을 이용한 이진 트리의 표현 - 편향 트리인 경우



<그림23_SW문제해결 기본_Tree_배열을 이용한 이진 트리의 표현 3>



<그림24_SW문제해결 기본_Tree_배열을 이용한 이진 트리의 표현 4>

SAMSUNG SW AI ACADEMY FOR YOUTH

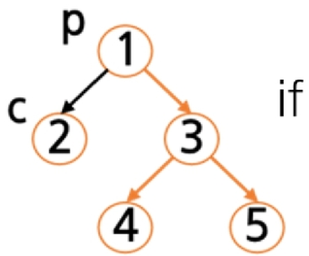
함께가요 미래로!
Enabling People

이진 트리 표현 2

부모 번호를 인덱스로 자식 번호를 저장

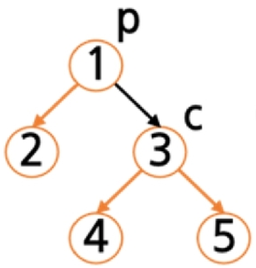
```
# 입력 예
5 4                    # 마지막 노드 V, 간선의 개수 N
1 2 1 3 3 4 3 5        # 부모 자식 순
```

부모	p	0	1	2	3	4	5
자식1	c1	0	2	0	0	0	0
자식2	c2	0	0	0	0	0	0



if c1[p] == 0
c1[p] = c

부모	p	0	1	2	3	4	5
자식1	c1	0	2	0	0	0	0
자식2	c2	0	3	0	0	0	0



else
c2[p] ← c

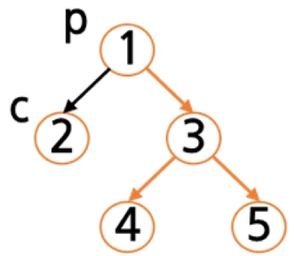
```
for i : 1 -> N
    read p, c
    if c1[p] == 0
        c1[p] = c
    else
        c2[p] = c
```

<그림25_SW문제해결 기본_Tree_부모 번호를 인덱스로 자식 번호를 저장>

자식 번호를 인덱스로 부모 번호를 저장

자식	c	0	1	2	3	4	5
부모	par	0	0	1	0	0	0

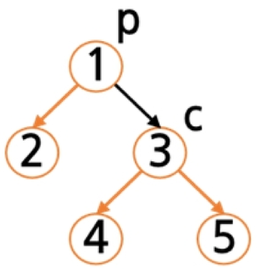
par[c] ← p




```
for i : 1 -> N
    read p, c
    par[c] = p
```

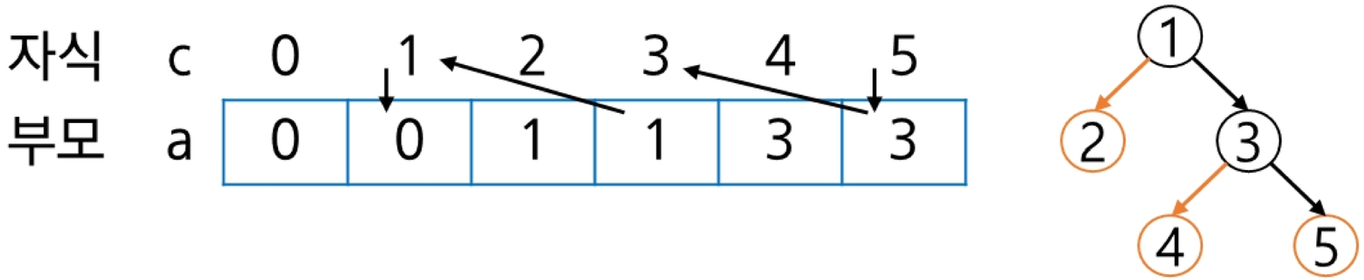
자식	c	0	1	2	3	4	5
부모	par	0	0	1	1	0	0

par[c] ← p



<그림26_SW문제해결 기본_Tree_자식 번호를 인덱스로 부모 번호를 저장>

루트 찾기, 조상 찾기



<그림27_SW문제해결 기본_Tree_루트 찾기, 조상 찾기>

ex) 5번 노드의 조상 찾기

```
c = 5
while( a[c] != 0 ) #루트인지 확인
    c = a[c]
    anc.append(c)  # 조상 목록
root = c
```

배열을 이용한 이진 트리의 표현의 단점

- 편향 이진 트리의 경우에 사용하지 않는 배열 원소에 대한 메모리 공간 낭비 발생
- 트리의 중간에 새로운 노드를 삽입하거나 기존의 노드를 삭제할 경우 배열의 크기 변경 어려워 비효율적

연결리스트를 이용한 트리의 표현

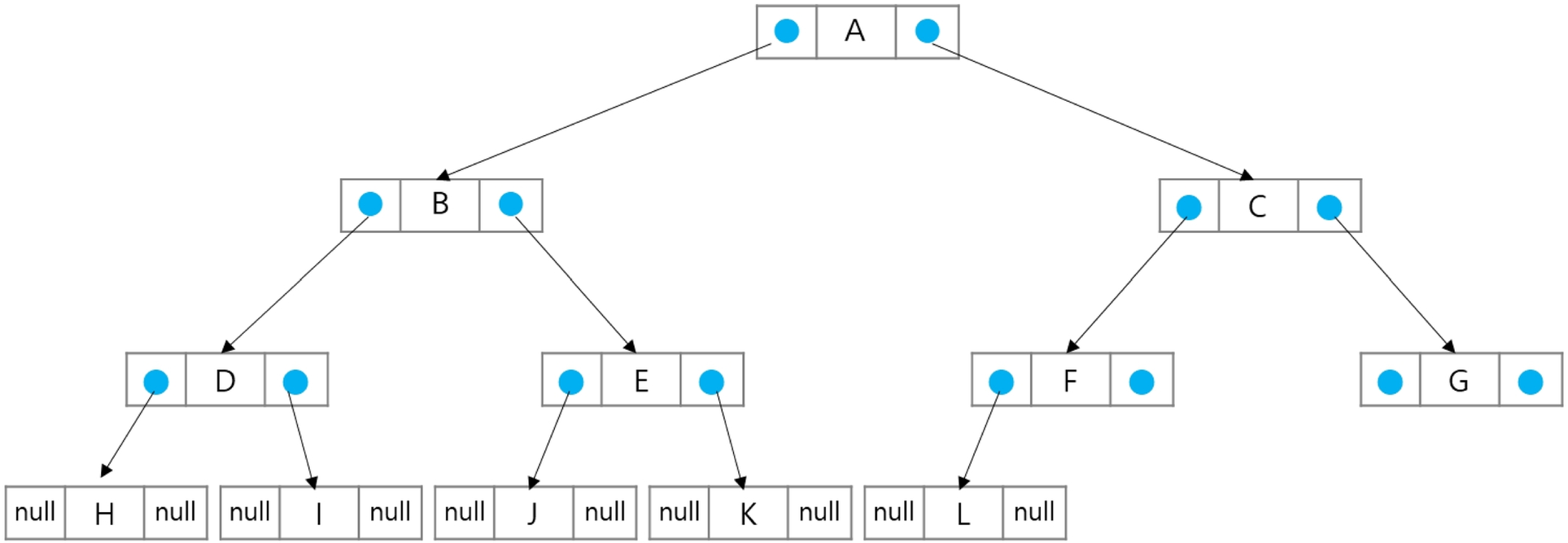
- 배열을 이용한 이진 트리의 표현의 단점을 보완

연결 자료구조를 이용한 이진 트리의 표현

- 이진 트리의 모든 노드는 최대 2개의 자식 노드를 가지므로 일정한 구조의 단순 연결 리스트 노드를 사용

<그림28_SW문제해결 기본_Tree_연결 자료구조를 이용한 이진 트리의 표현>

완전 이진 트리의 연결 리스트 표현



<그림29_SW문제해결 기본_Tree_완전 이진 트리의 연결 리스트 표현>

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

순회

순회

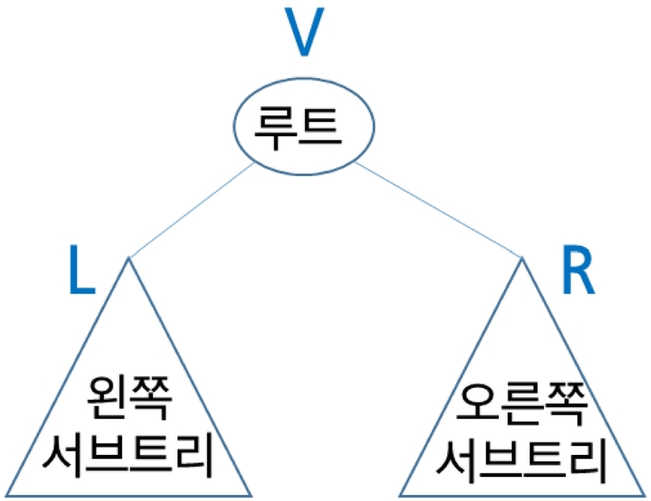
순회

트리의 각 노드를 중복되지 않게 전부 방문(visit)하는 것

트리는 비선형 구조이기 때문에 선형 구조에서와 같이
선후 연결 관계를 알 수 없습니다.

순회 (Traversal)

- 트리의 노드들을 체계적으로 방문하는 것



<그림13_SW문제해결 기본_Tree_순회>

3가지의 기본적인 순회 방법

- 전위 순회(Preorder Traversal) : VLR
  - 부모 노드 방문 후, 자식 노드를 좌, 우 순서로 방문합니다.
- 중위 순회(Inorder Traversal) : LVR
  - 왼쪽 자식 노드, 부모 노드, 오른쪽 자식 노드 순으로 방문합니다.
- 후위 순회(Postorder Traversal) : LRV
  - 자식 노드를 좌, 우 순서로 방문한 후, 부모 노드로 방문합니다.

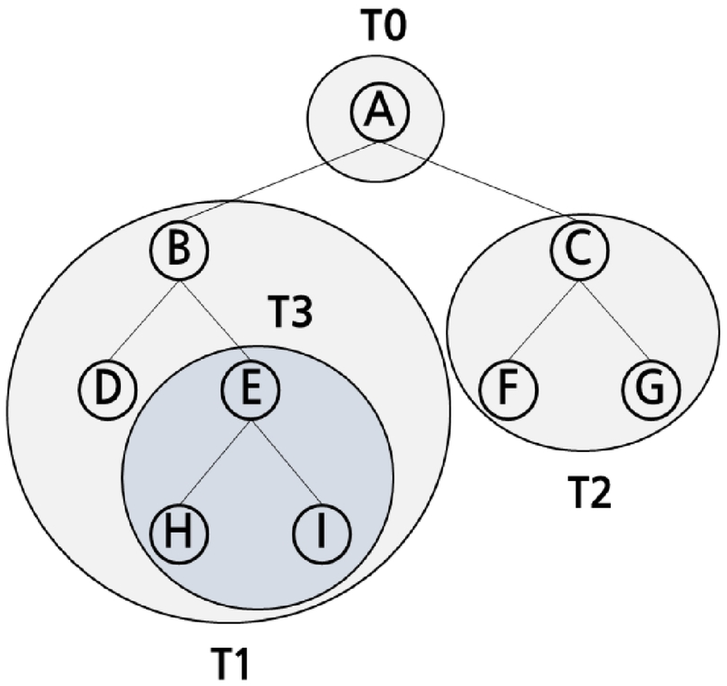
전위 순회(Preorder Traversal)

- **수행 방법**
  ① 현재 노드 n을 방문하여 처리한다. → V
  ② 현재 노드 n의 왼쪽 서브트리로 이동한다. → L
  ③ 현재 노드 n의 오른쪽 서브트리로 이동한다. → R

- **전위 순회 알고리즘**

```
preorder_traverse(T) :                  # 전위 순회
    if T :                              # T is not None
        visit(T)                        # print(T.item)
        preorder_traverse(T.left)
        preorder_traverse(T.right)
```

전위 순회의 예



<그림14_SW문제해결 기본_Tree_전위 순회의 예>

순서1 : T0 → T1 → T2

순서2 : A → B D (T3) → C F G

총 순서 : A B D E H I C F G

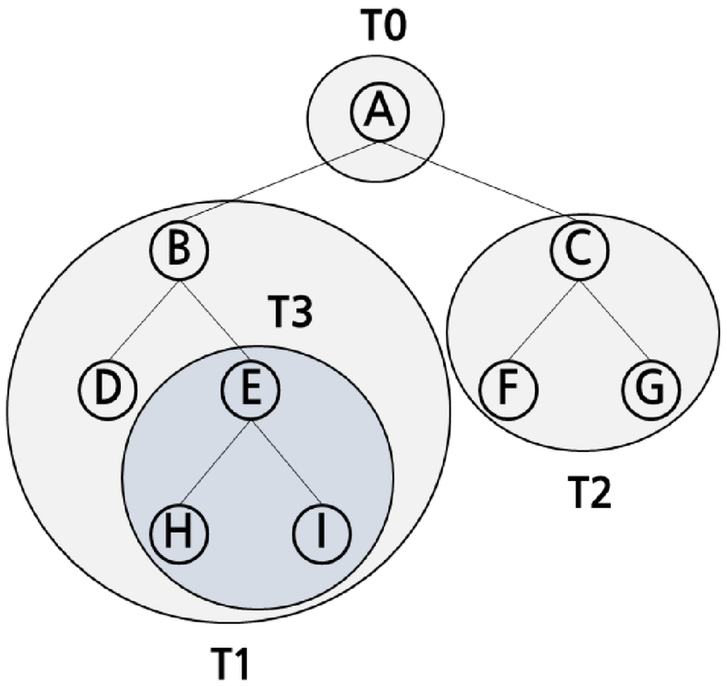
중위 순회 (Inorder Traversal)

- **수행 방법**
  ① 현재 노드 n의 왼쪽 서브트리로 이동한다. → L
  ② 현재 노드 n을 방문하여 처리한다. → V
  ③ 현재 노드 n의 오른쪽 서브트리로 이동한다. → R

- **중위 순회 알고리즘**

```
inorder_traverse(T) :                    # 중위 순회
    if T :                               # T is not None
        inorder_traverse(T.left)
        visit(T)                         # print(T.item)
        inorder_traverse(T.right)
```

중위 순회의 예



<그림15_SW문제해결 기본_Tree_중위 순회의 예>

순서1 : T1 → T0 → T2

순서2 : D B (T3) → A → F C G

총 순서 : D B H E I A F C G

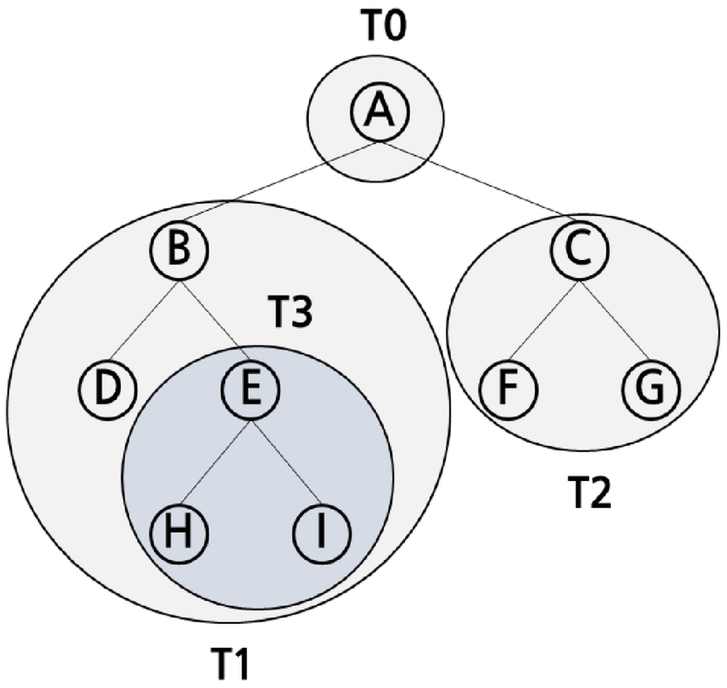
후위 순회(Postorder Traversal)

- **수행 방법**
  ① 현재 노드 n의 왼쪽 서브트리로 이동한다. → L
  ② 현재 노드 n을 오른쪽 서브트리로 이동한다. → R
  ③ 현재 노드 n을 방문하여 처리한다. → V

- **후위 순회 알고리즘**

```
postorder_traverse(T) :                    # 후위 순회
    if T :                                 # T is not None
        postorder_traverse(T.left)
        postorder_traverse(T.right)
        visit(T)                           # print(T.item)
```

후위 순회의 예



<그림16_SW문제해결 기본_Tree_후위 순회의 예>

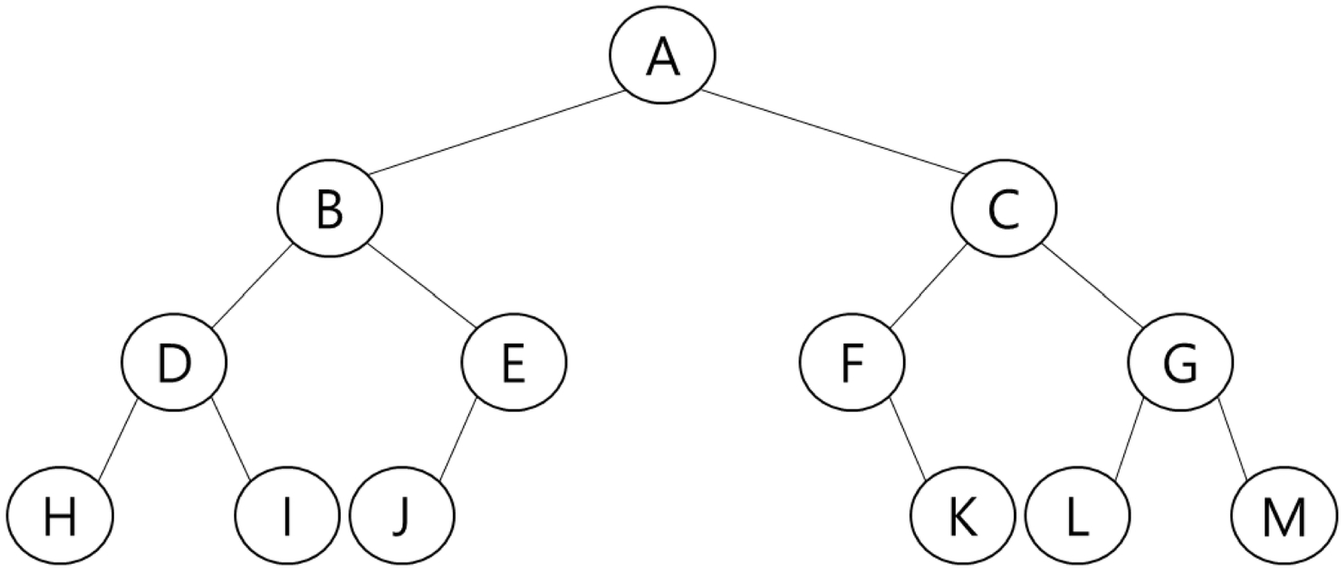
순서1 : T1 → T2 → T0

순서2 : D (T3) B → F G C → A

총 순서 : D H I E B F G C A

이진 트리의 순회

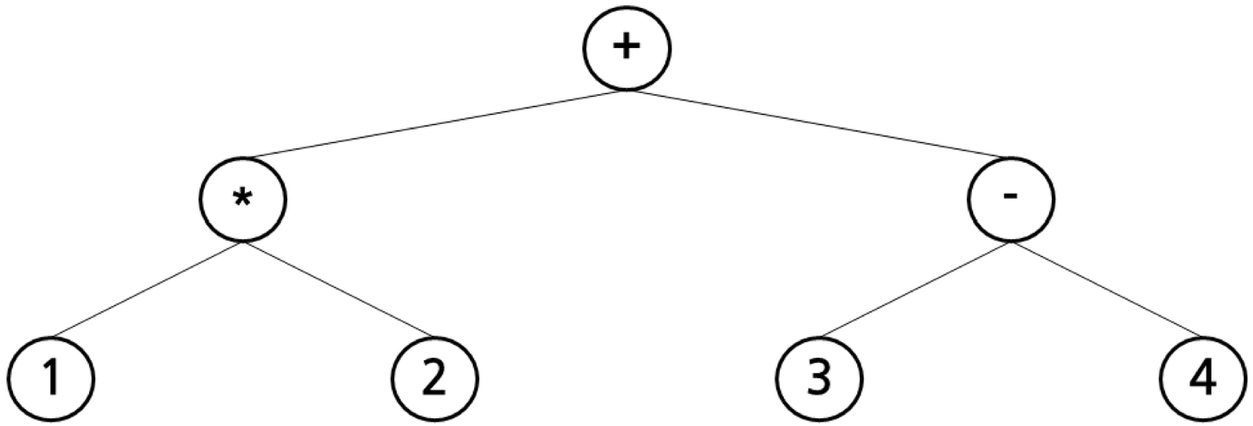
- 전위 순회는?
- 중위 순회는?
- 후위 순회는?



<그림17_SW문제해결 기본_Tree_이진 트리의 순회>

수식 트리

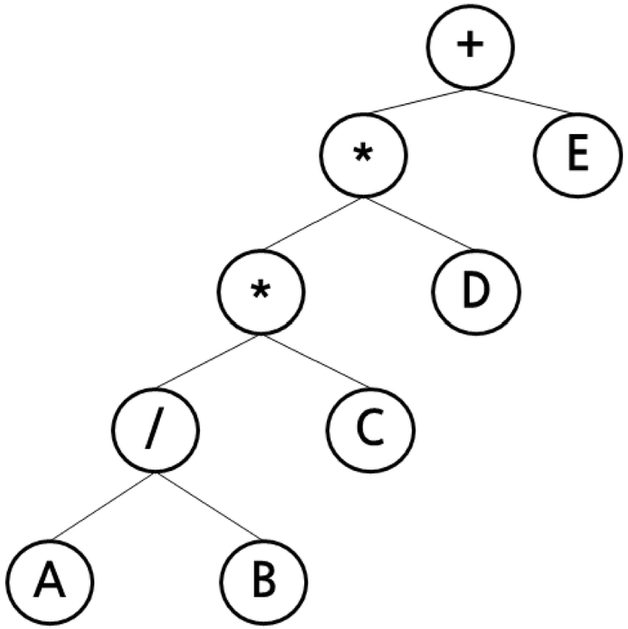
- 수식을 표현하는 이진 트리
- 수식 이진 트리 (Expression Binary Tree)라고 부르기도 함
- 연산자는 루트 노드이거나 가지 노드
- 피연산자는 모두 잎 노드



<그림30_SW문제해결 기본_Tree_수식 트리>

수식 트리의 순회

- 중위 순회
  - A / B * C * D + E (식의 중위 표기법)
- 후위 순회
  - A B / C * D * E + (식의 후위 표기법)
- 전위 순회
  - + * * / A B C D E (식의 전위 표기법)



<그림31_SW문제해결 기본_Tree_수식 트리의 순회>

연습 문제

전위 순회

- 전위 순회하여 정점의 번호를 출력
  - 첫 줄에는 트리의 정점의 총 수 V, 그 다음 줄에는 V-1개 간선 정보가 나열됩니다.
  - 간선은 그것을 이루는 두 정점으로 표기됩니다.
  - 간선은 항상 “부모 자식” 순서로 표기되며, 아래 예에서 두 번째 줄 처음 1과 2는 정점 1과 2를 잇는 간선으로 1이 부모, 2가 자식을 의미합니다.
- 코드 : 7_Tree/ex.py

13

1 2 1 3 2 4 3 5 3 6 4 7 5 8 5 9 6 10 6 11 7 12 11 13

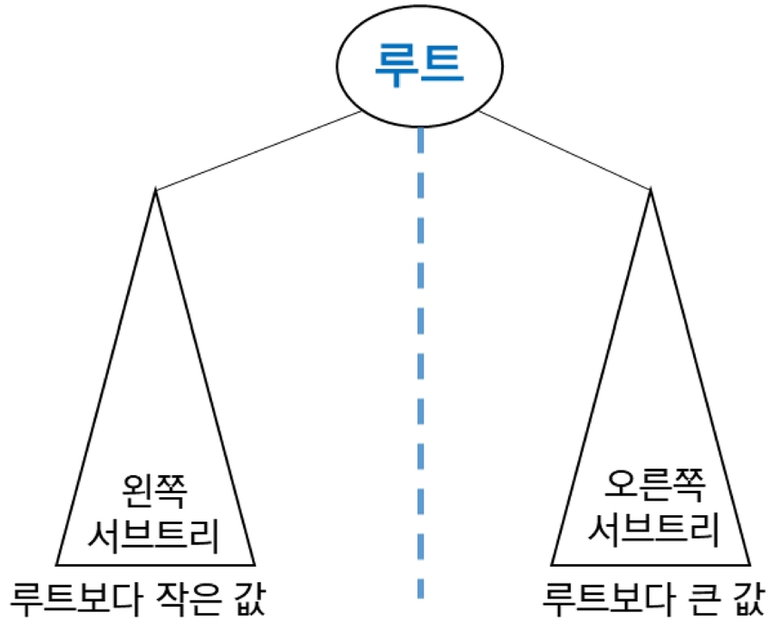
SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

이진 탐색 트리

이진 탐색 트리 (BST, Binary Search Tree)

- 데이터를 빠르게 검색할 수 있도록 체계적으로 저장
- 최대 O(log n)의 빠른 속도로 값을 검색할 수 있는 자료구조
- 빠르게 검색될 수 있도록, 특정 규칙을 갖는 이진 트리 형태로 값을 저장



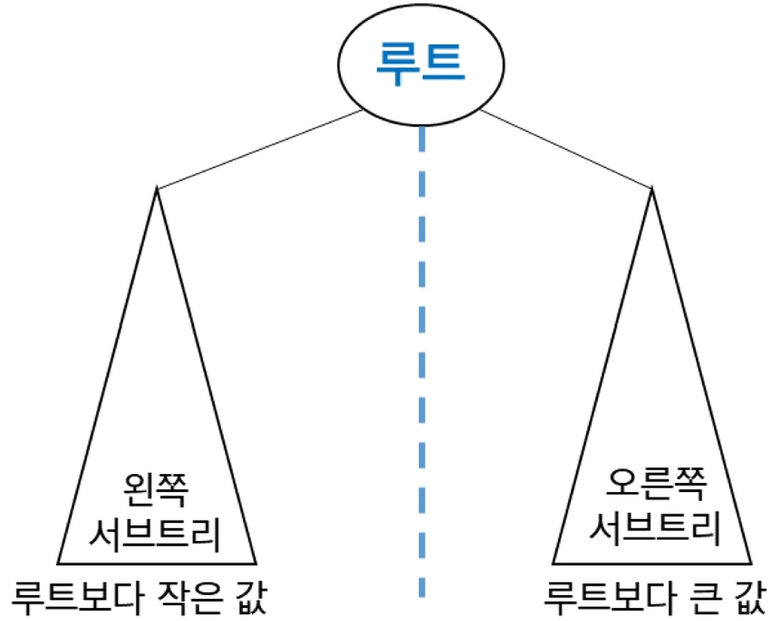
<그림32_SW문제해결 기본_Tree_이진 탐색 트리>

연결리스트 vs BST

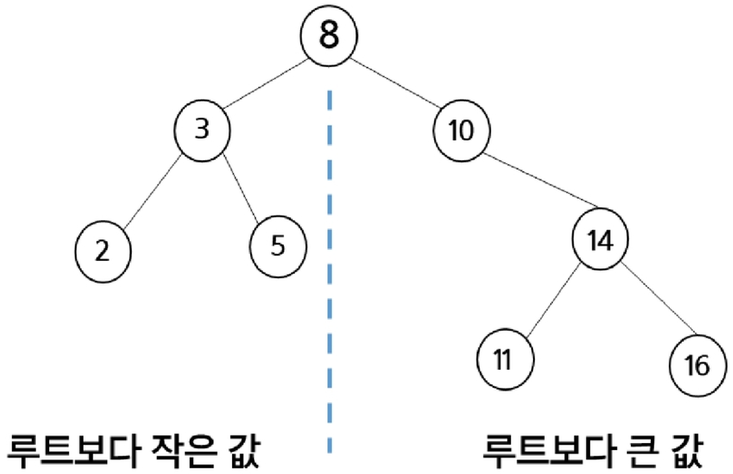
- BST는 선형 연결리스트 보다 더 빠른 삽입 / 삭제 / 탐색이 가능함
- 연결리스트 성능
  - 삽입 : O(N)
  - 삭제 : O(N)
  - 탐색 : O(N)
- BST 성능
  - 삽입 : 평균 O(log N)
  - 삭제 : 평균 O(log N)
  - 탐색 : 평균 O(log N)

이진 탐색 트리 (BST, Binary Search Tree)

- 탐색 작업을 효율적으로 하기 위한 자료구조
- 모든 원소는 서로 다른 유일한 키를 가짐
- key (왼쪽 서브트리) < key (루트 노드) < key (오른쪽 서브트리)
- 왼쪽 서브트리와 오른쪽 서브트리도 이진 탐색 트리임
- 중위 순회하면 오름차순으로 정렬된 값을 얻을 수 있음



<그림32_SW문제해결 기본_Tree_이진 탐색 트리>



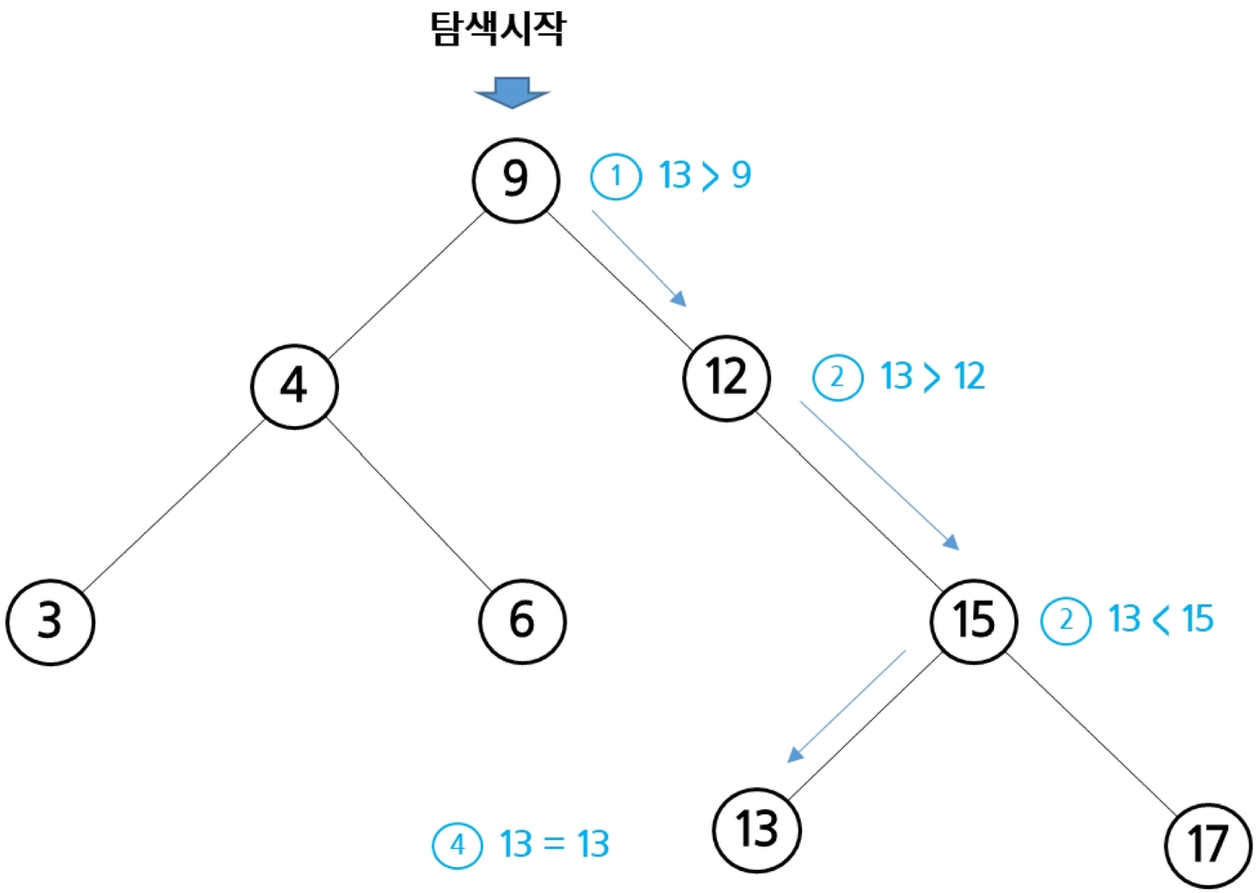
<그림33_SW문제해결 기본_Tree_이진 탐색 트리의 예시>

탐색 연산

- 루트에서 시작
- 탐색할 키 값 x를 루트 노드의 키 값과 비교함
  - (키 값 x = 루트 노드의 키 값)인 경우 : 원하는 원소를 찾았으므로 탐색 연산에 성공한 경우입니다.
  - (키 값 x < 루트 노드의 키 값)인 경우 : 루트 노드의 왼쪽 서브트리에 대해서 탐색 연산을 수행합니다.
  - (키 값 x > 루트 노드의 키 값)인 경우 : 루트 노드의 오른쪽 서브트리에 대해서 탐색 연산을 수행합니다.
- 서브트리에 대해서 순환적으로 탐색 연산을 반복함

탐색 연산

- 13 탐색



<그림34_SW문제해결 기본_Tree_탐색 연산>

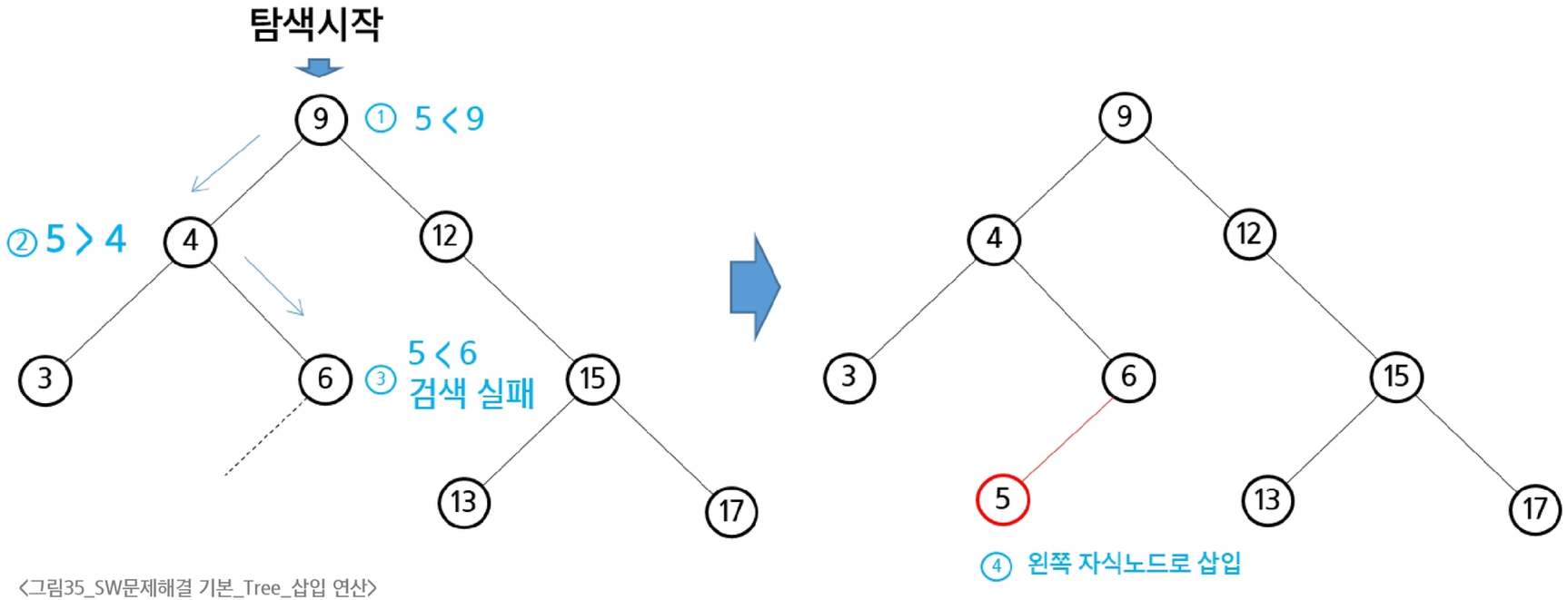
삽입 연산

① 먼저 탐색 연산을 수행

- 삽입할 원소와 같은 원소가 있으면 삽입할 수 없으므로, 같은 원소가 있는지 탐색 합니다.

② 탐색 실패한 위치에 원소를 삽입

- 다음은 5를 삽입하는 예시 입니다.



<그림35_SW문제해결 기본_Tree_삽입 연산>

이진 탐색 트리의 성능

- 탐색(searching), 삽입(insertion), 삭제(deletion) 시간은 트리의 높이만큼 시간이 걸림
  - O(h), h : BST의 깊이 (height)
- 평균의 경우
  - 이진 트리가 균형적으로 생성되어 있는 경우입니다.
  - O(log n) 시간에 탐색할 수 있습니다.
- 최악의 경우
  - 한쪽으로 치우친 경사 이진 트리의 경우입니다.
  - O(n) 시간이 걸립니다.
  - 순차탐색과 시간 복잡도가 같습니다.

이진 탐색 트리의 성능

- 이진 탐색 트리에서의 평균 : O(logN)

- 최악의 경우 O(N) 입니다.
- 완전 이진 트리 또는 균형 트리로 바꾸면 최악의 경우를 없앨 수 있고, 삽입 삭제 시간이 줄어듭니다.
- 평균과 최악의 시간이 O(logN)으로 같습니다.

다른 검색의 성능

- 배열에서의 순차 검색 : O(N)
- 정렬된 배열에서의 순차 검색 : O(N)
- 정렬된 배열에서의 이진 탐색 : O(logN)

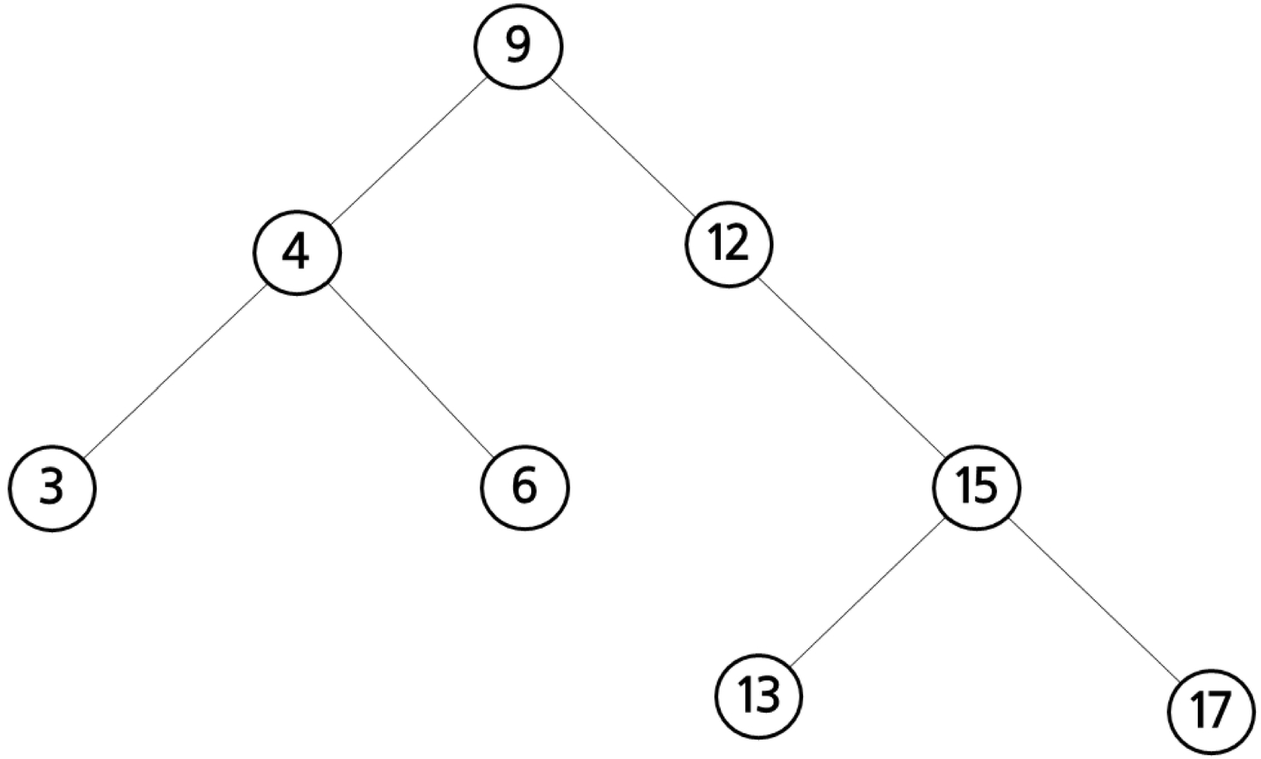
- 크기가 고정된 배열에서 삽입, 삭제 시 추가 연산이 필요합니다.

- 해쉬 검색 : O(1)

- 추가 저장 공간이 필요합니다.

삭제 연산

- 삭제 연산에 대해 알고리즘을 생각해 보기
- 13, 12, 9를 차례로 삭제한다면?



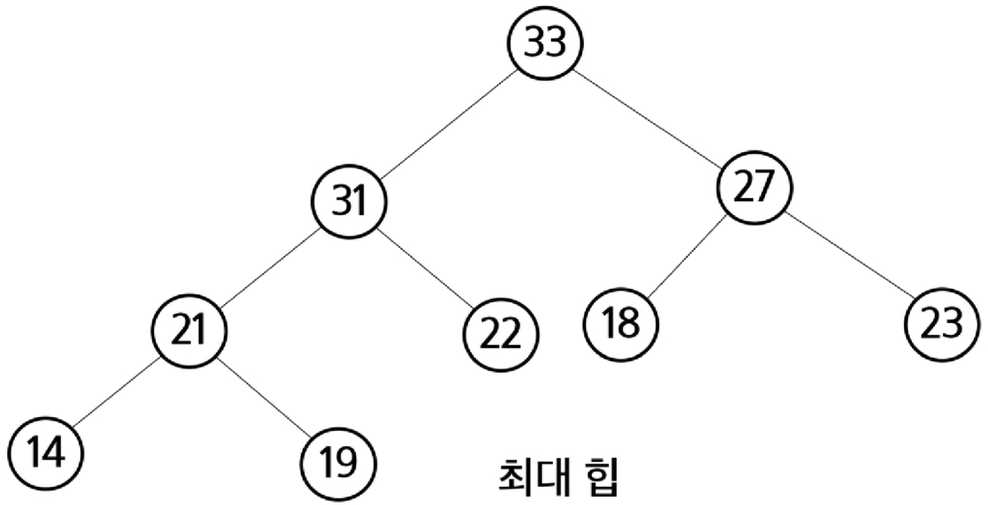
<그림36_SW문제해결 기본_Tree_삭제 연산>

힙

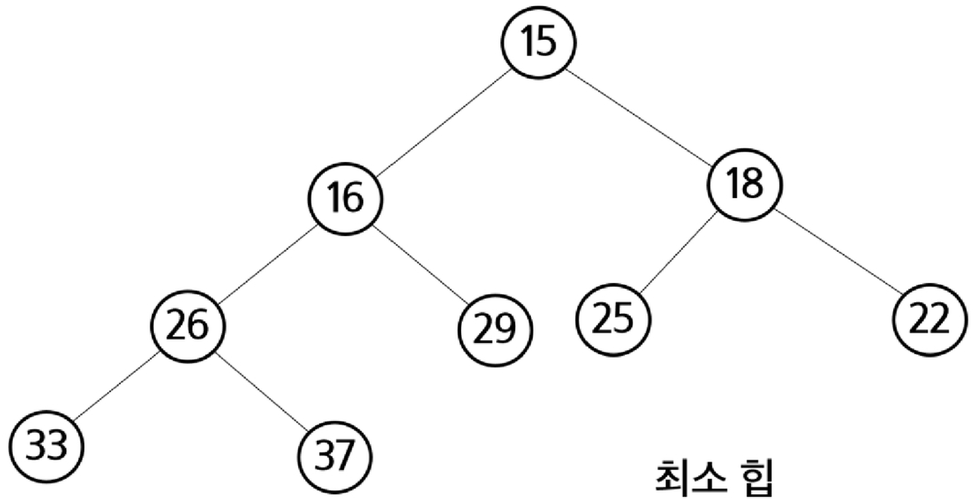
힙(Heap)

- 완전 이진 트리에 있는 노드 중에서 키 값이 가장 크거나 작은 노드를 찾기 위해서 만든 자료구조
- 최대 힙(Max Heap)
  - 키 값이 가장 큰 노드를 찾기 위한 힙 입니다.
  - 항상 {부모 노드의 키 값 > 자식 노드의 키 값} 조건을 만족합니다.
  - 루트 노드 : 키 값이 가장 큰 노드입니다.
- 최소 힙(Min Heap)
  - 키 값이 가장 작은 노드를 찾기 위한 힙 입니다.
  - 항상 {부모 노드의 키 값 < 자식 노드의 키 값} 조건을 만족합니다.
  - 루트 노드 : 키 값이 가장 작은 노드입니다.

힙의 예

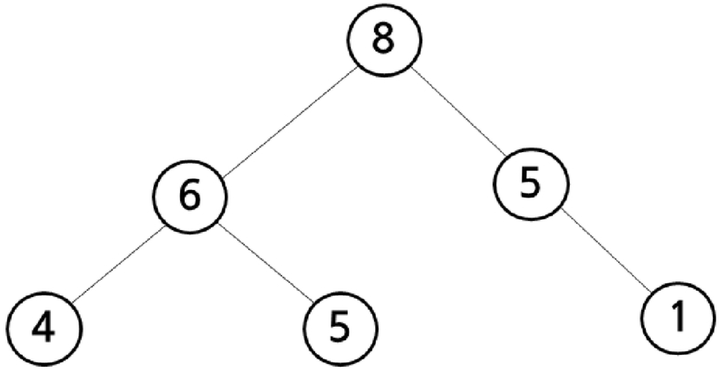


<그림37_SW문제해결 기본_Tree_최대 힙>



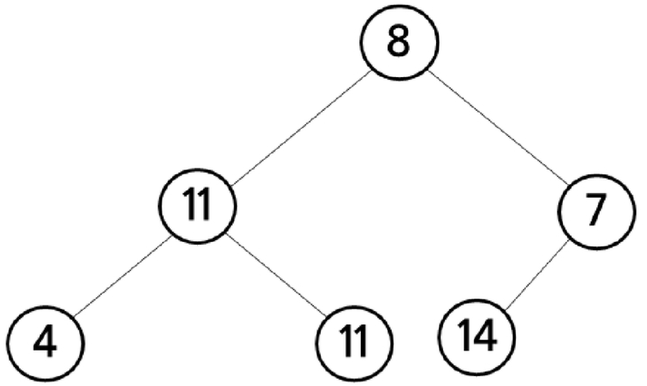
<그림38_SW문제해결 기본_Tree_최소 힙>

힙이 아닌 이진 트리의 예



트리1

<그림39_SW문제해결 기본_Tree_힙이 아닌 이진 트리의 예 1>

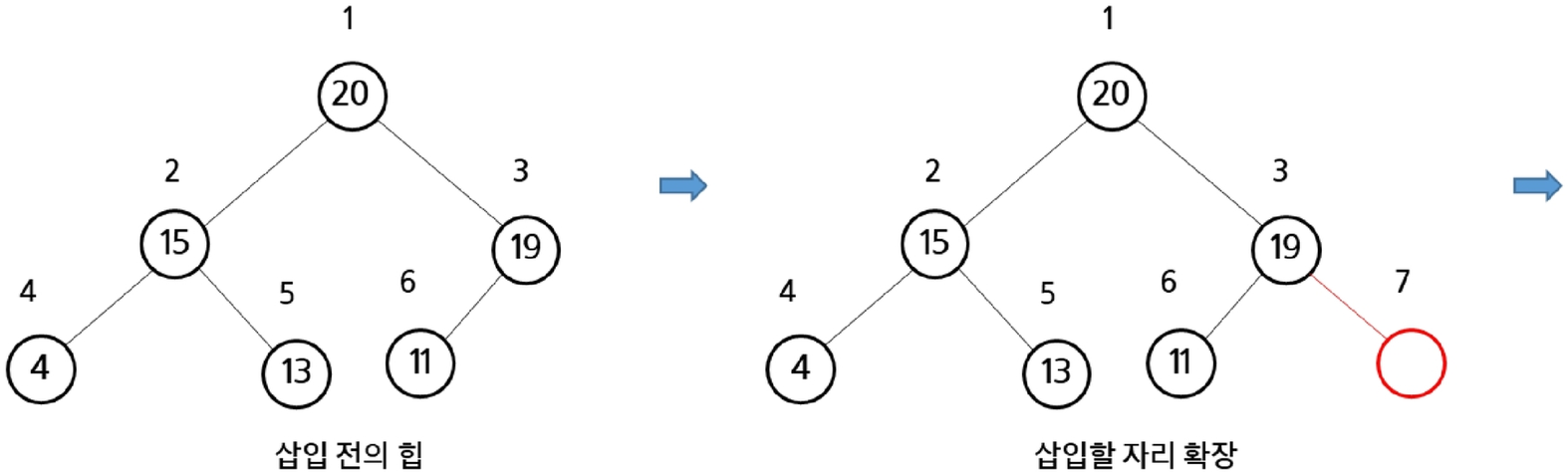


트리2

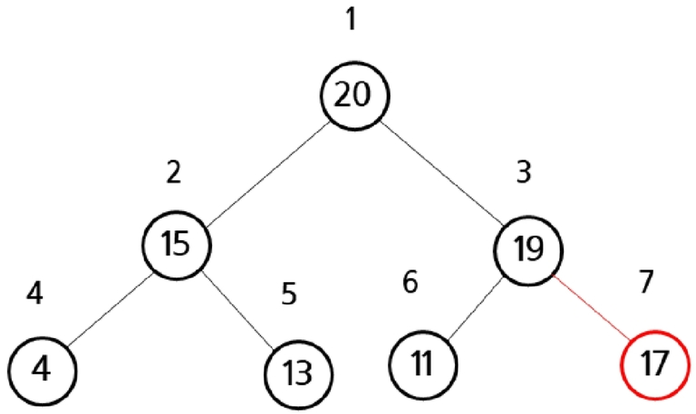
<그림40_SW문제해결 기본_Tree_힙이 아닌 이진 트리의 예 2>

힙 연산 - 삽입

- 17 삽입



삽입 전의 힙 　　삽입할 자리 확장

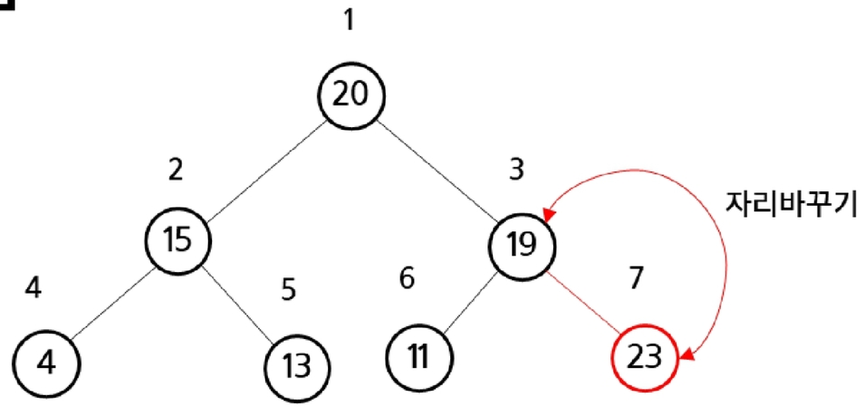


확장한 자리에 삽입할 원소 저장

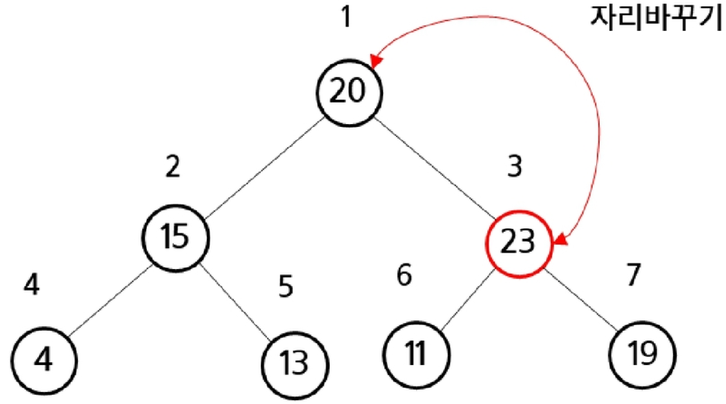
<그림41_SW문제해결 기본_Tree_17 삽입>

힙 연산 - 삽입

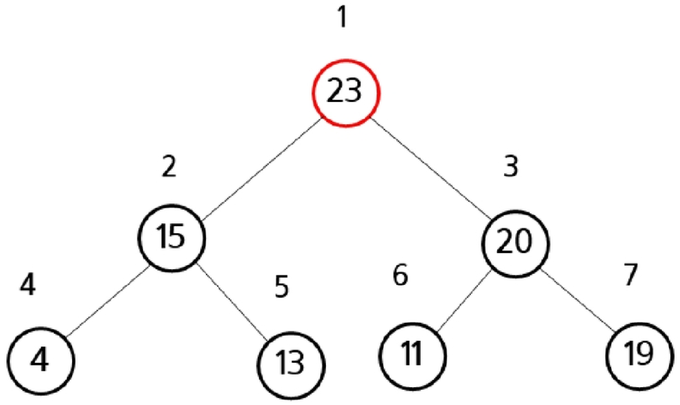
- 23 삽입



1) (삽입 노드 23 > 부모 노드 19) : 자리바꾸기



2) (삽입 노드 23 > 부모 노드 19) : 자리바꾸기



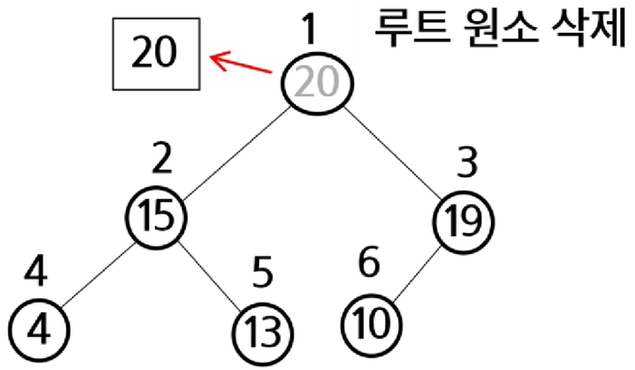
3) 비교할 부모 노드가 없으므로 자리 확정

<그림42_SW문제해결 기본_Tree_23 삽입>

힙 연산 - 삭제

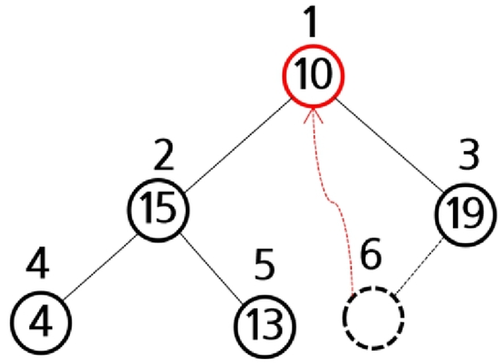
- 힙에서는 루트 노드의 원소만을 삭제할 수 있음
- 루트 노드의 원소를 삭제하여 반환함
- 힙의 종류에 따라 최댓값 또는 최솟값을 구할 수 있음

힙 연산 - 삭제

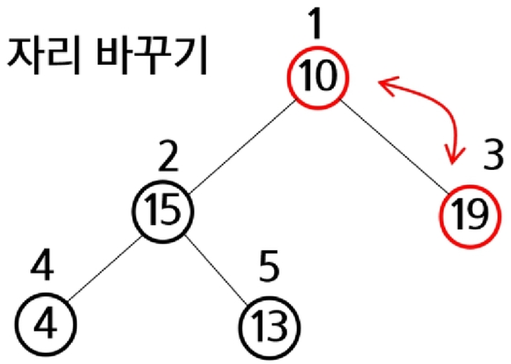


루트 원소 삭제

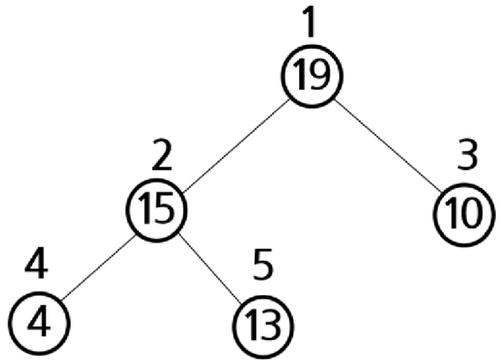
1) 루트 노드 백업 후 원소 삭제



2) 마지막 노드 원소를 루트에 복사 후 삭제



3) 자리 바꾸기 (10 < 19)

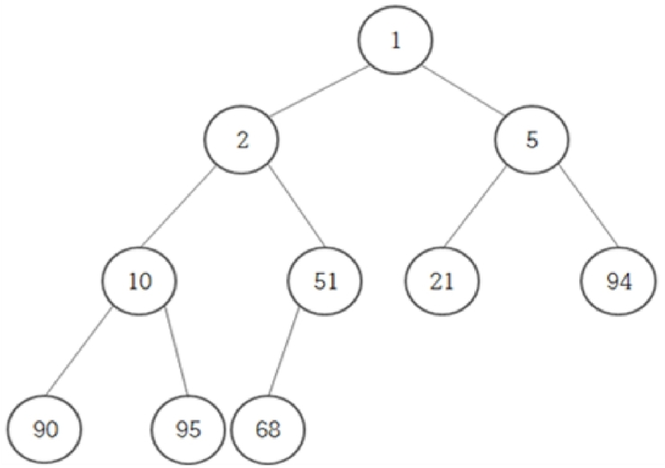


4) 자리 확정

<그림43_SW문제해결 기본_Tree_힙 연산-삭제>

힙을 이용한 우선순위 큐

- 완전 이진 트리로 구현된 자료구조로서, 키 값이 가장 큰 노드나 가장 작은 노드를 찾기에 적합한 자료구조
- 아래의 예는 최소 힙(Min Heap) 으로서, 가장 작은 키 값을 가진 노드가 항상 루트에 위치



<최소 힙의 예>

<그림44_SW문제해결 기본_Tree_최소 힙의 예>

- 힙의 키를 우선순위로 활용하여 우선순위 큐를 구현
- 코드 : 7_Tree/max_heap.py

실습

순회

- 5174.subtree
- 5176.이진탐색

힙

- 5177.이진 힙

완전이진트리

- 5178.노드의 합

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

1 트리의 정의에 대한 설명으로 올바른 것은?

a) 모든 노드는 정확히 두 개의 자식 노드를 가진다.
b) 트리는 사이클이 존재할 수 있는 자료구조이다.
c) 트리는 계층적이며 1:N 관계를 갖는 비선형 자료구조이다.
d) 트리는 항상 이진 탐색 트리 구조를 따라야 한다.

2 트리의 용어 중 '리프 노드'는 무엇을 의미하는가?

a) 부모 노드가 없는 노드
b) 자식 노드를 모두 갖고 있는 노드
c) 자식 노드가 하나만 있는 노드
d) 자식 노드가 없는 노드

3 다음 중 루트 노드에 대한 설명으로 적절한 것은?

a) 항상 차수가 0이다.
b) 트리에서 가장 마지막에 생성된 노드
c) 부모 노드가 존재하지 않는 노드
d) 항상 형제 노드를 가진다.

4 이진 트리에서 각 노드가 가질 수 있는 최대 자식 노드 수는?

a) 1
b) 2
c) 3
d) 제한 없음

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

5 높이 h인 포화 이진 트리의 노드 수는?

a) h+1
b) 2^h
c) $2^{(h+1)}-1$
d) h^2

6 다음 중 이진 탐색 트리(BST)의 특징으로 올바른 것은?

a) 항상 완전 이진 트리 구조이다.
b) 부모 노드보다 왼쪽 자식은 항상 크다.
c) 중복된 값을 허용한다.
d) 왼쪽 서브트리 < 루트 < 오른쪽 서브트리

7 중위 순회(inorder traversal)의 방문 순서는?

a) 부모 → 왼쪽 → 오른쪽
b) 왼쪽 → 부모 → 오른쪽
c) 오른쪽 → 부모 → 왼쪽
d) 부모 → 오른쪽 → 왼쪽

8 다음 중 최소 힙 (Min Heap)의 조건으로 옳은 것은?

a) 자식 노드 ≥ 부모 노드
b) 루트 노드가 항상 최대값을 가진다.
c) 완전 이진 트리이다.
d) 자식 노드가 없으면 트리가 아니다.

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

9 배열로 이진 트리를 표현할 때, i번 노드의 왼쪽 자식 노드 번호는?

a) i - 1
b) 2 * i
c) i + 1
d) i // 2

10 트리에서 '서브트리(subtree)'의 정의는?

a) 노드의 부모 노드만으로 구성된 트리
b) 루트 노드를 제외한 나머지 전체 노드
c) 특정 노드와 그 노드의 자손 노드들로 구성된 트리
d) 자식 노드가 없는 노드들의 모임

11 다음 중 트리의 전위 순회(preorder)의 순서는?

a) LVR
b) VRL
c) VLR
d) RVL

12 다음 중 트리를 연결리스트로 표현할 때의 장점은?

a) 삽입과 삭제가 어려워진다.
b) 모든 노드를 순차적으로 접근해야 한다.
c) 포화 트리에서만 사용할 수 있다.
d) 메모리 낭비를 줄이고 유동적인 크기를 가질 수 있다.

확인문제

정답 및 해설

1. c) 트리는 계층적이며 1:N 관계를 갖는 비선형 자료구조이다. 2. d) 자식 노드가 없는 노드 3. c) 부모 노드가 존재하지 않는 노드 4. b) 2 5. c) $2^{(h+1)}-1$ 6. d) 왼쪽 서브트리 < 루트 < 오른쪽 서브트리 7. b) 왼쪽 → 부모 → 오른쪽 8. c) 완전 이진 트리이다. 9. b) 2 * i 10. c) 특정 노드와 그 노드의 자손 노드들로 구성된 트리 11. c) VLR 12. d) 메모리 낭비를 줄이고 유동적인 크기를 가질 수 있다.

1. 트리는 부모-자식 간 1:N 관계를 가지는 비선형 자료구조입니다.

2. 리프 노드는 자식 노드가 하나도 없는, 가장 말단에 위치한 노드입니다.

3. 루트 노드는 트리에서 최상단에 위치하며 부모 노드를 가지지 않습니다.

4. 이진 트리는 각 노드가 최대 두 개의 자식 노드를 가질 수 있는 구조입니다.

5. 포화 이진 트리는 모든 레벨이 가득 찼을 때의 노드 수가 2^(h+1)-1 입니다.

6. BST는 왼쪽 자식 < 루트 < 오른쪽 자식이라는 키 값 규칙을 따릅니다.

7. 중위 순회는 왼쪽 → 부모 → 오른쪽 순서로 노드를 방문합니다.

8. 힙은 반드시 완전 이진 트리 구조를 따라야 힙 조건이 성립합니다.

9. 배열 기반 이진 트리에서는 왼쪽 자식 노드는 2 * i로 계산됩니다.

10. 서브트리는 특정 노드를 루트로 하여 그 자식까지 포함하는 하위 트리입니다.

11. 전위 순회는 루트 → 왼쪽 자식 → 오른쪽 자식 순서입니다.

12. 연결 리스트 구조는 배열보다 메모리를 효율적으로 사용할 수 있고 확장성이 좋습니다.

활동정리

핵심키워드

개념	설명	예시
루트 노드 (Root Node)	트리의 최상위 노드	가계도에서 가장 위 조상
자식 노드 (Child Node)	부모 노드로부터 연결된 하위 노드	부모 자식 관계에서 자식
리프 노드 (Leaf Node)	자식이 없는 노드 (차수 0)	가계도에서 가장 아래
서브트리 (Subtree)	특정 노드를 루트로 하는 하위 트리	전체 가계도에서 한 명을 기준으로 모든 자손을 표시한 가계도
순회 (Traversal)	트리의 모든 노드를 한 번씩 방문하는 방법	전위, 중위, 후위 순회
이진 트리 (Binary Tree)	노드당 최대 두 개의 자식 노드를 가지는 트리	이진 탐색 트리
힙 (Heap)	우선순위 기반의 완전 이진 트리	우선 순위 큐

<표1_SW문제해결 기본_Tree_핵심 키워드>

활동정리

오늘 공부한 내용 요약 및 정리

- **트리**
  - 원소 간 계층적 관계를 가지는 비선형(1:N 관계) 자료구조
  - 하나의 루트에서 시작해 여러 자식 노드로 분기하는 구조

- **트리의 구성 요소**
  - 루트 노드 : 최상위 노드, 부모가 없음
  - 형제 노드 : 계층 구조에서 같은 부모를 공유
  - 리프 노드 : 자식이 없는 노드 (= 단말 노드)
  - 서브트리 : 특정 노드를 루트로 하는 하위 트리

- **트리의 특징**
  - 차수 : 노드가 가진 자식 노드의 수
  - 높이 : 루트에서 가장 깊은 노드까지의 거리 (= 최대 레벨)

활동정리

오늘 공부한 내용 요약 및 정리

- **이진 트리**
  - 모든 노드가 최대 두 개의 자식 노드를 갖는 트리

- **이진 탐색 트리**
  - 왼쪽 서브 트리 키 < 루트 키 < 오른쪽 서브 트리 키 규칙을 따름
  - 탐색, 삽입, 삭제가 평균 O(log n)의 시간 복잡도를 가짐
  - 중위 순회 시 오름차순 정렬된 결과 출력

- **힙 이란?**
  - 완전 이진 트리 기반의 우선순위 큐 자료구조
  - 종류 : 최대 힙, 최소 힙
  - 삽입/삭제 시 재정렬 필요

활동정리

자료구조를 이용해 가계도를 표현해 봅시다!

1. 부모와 자식 관계를 표현할 수 있는 [ ]자료구조가 필요합니다.
2. [ ] 자료구조인 [ ]를 이용해 표현할 수 있습니다.

활동정리

자료구조를 이용해 가계도를 표현해 봅시다!

1. 부모와 자식 관계를 표현할 수 있는 비선형 자료구조가 필요합니다.
2. 비선형 자료구조인 트리를 이용해 표현할 수 있습니다.

```
# 조부모, 부모, 고모, 나, 동생, 1명인 사촌을 완전 이진 트리로 저장
tree = [0, '조부모', '고모', '부모', '고종사촌', 0, '나', '동생']
```

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

감사합니다